

NUnit Unit Testing Hands-On

Objective: Write all possible unit tests for the ParseHostName method in the UrlHostNameParser class using NUnit. The method has two execution paths, so we need at least two test methods. We apply the Single Assertion Rule and use Assert.That() throughout.

Step 1 — Source Project: UtilLib

Project Structure

```
Solution: UtilLib.sln
|   +-- UtilLib/                               (Class Library - source
project) |   UrlHostNameParser.cs
|   +-- UtilLib.Tests/                         (Class Library - test
project)   UrlHostNameParserTests.cs
```

UtilLib / UrlHostNameParser.cs — Source Code Under Test

```
namespace UtilLib
{
    public class UrlHostNameParser
    {
        /// <summary>          /// Parses the host name
        from a full URL string.
        /// Execution Path 1: URL starts with "www." → strips scheme + "www."
        /// Execution Path 2: URL does NOT start with "www." → strips scheme only
        /// </summary>
        public string ParseHostName(string url)
        {
            // Remove the scheme (http:// or
            https://)          string withoutScheme = url;

            if (url.Contains("://"))
            {
                withoutScheme = url.Substring(url.IndexOf("://") + 3);
            }

            // Remove trailing path after the host (everything after first '/')
            int slashIndex = withoutScheme.IndexOf('/');          if (slashIndex >= 0)
            {
                withoutScheme = withoutScheme.Substring(0, slashIndex);
            }

            // Execution Path 1: host starts with "www." - strip it
            if (withoutScheme.StartsWith("www."))          {
                return withoutScheme.Substring(4);          // strip "www."
            }

            // Execution Path 2: no "www." prefix - return host as-is
            return withoutScheme;          }
        }
    }
}
```

Two execution paths identified: Path 1 — URL contains 'www.' e.g. http://www.google.com → returns 'google.com' Path 2 — URL has no 'www.' e.g. http://google.com → returns 'google.com'

Step 2 — Create and Configure the Test Project: UtilLib.Tests

Project Setup in Visual Studio

1. Right-click on Solution → Add → New Project
2. Choose: Class Library (.NET)

```
3. Name: UtilLib.Tests
```

```
4. Rename Class1.cs → UrlHostNameParserTests.cs Install
```

NuGet Packages (Package Manager Console)

```
# Make sure UtilLib.Tests is selected as the target project
```

```
Install-Package NUnit
```

```
Install-Package NUnit3TestAdapter
```

```
Install-Package Microsoft.NET.Test.Sdk
```

```
Successfully installed 'NUnit 3.14.0' to UtilLib.Tests
```

```
Successfully installed 'NUnit3TestAdapter 4.5.0' to UtilLib.Tests
```

```
Successfully installed 'Microsoft.NET.Test.Sdk' to UtilLib.Tests
```

Add Reference to UtilLib

```
# Right-click UtilLib.Tests → Add → Project Reference
```

```
# Tick: UtilLib → OK
```

```
# The test project can now access UrlHostNameParser.
```

```
Reference added:
```

```
UtilLib.Tests references UtilLib ✓
```

Step 3 — Write the Test Class: UrlHostNameParserTests.cs

Naming Conventions Used

Test Project : UtilLib.Tests Test Class : UrlHostNameParserTests (SUT = UrlHostNameParser) Test Methods :

UnitUnderTest_Scenario_ExpectedOutcome Rule : One Assert.That() per test method (Single Assertion Rule)

UrlHostNameParserTests.cs — Full Test Class

```
using NUnit.Framework;
using UtilLib;

namespace UtilLib.Tests
{
    [TestFixture]
    public class UrlHostNameParserTests
    {
        private UrlHostNameParser _parser;

        [SetUp]
        public void Setup()
        {
            // Create a fresh instance before every test
            _parser = new UrlHostNameParser();
        }

        // Execution Path 1: URL with www. prefix

        [Test]
        public void ParseHostName_UrlWithWwwPrefix_ReturnsHostWithoutWww()
        {
            // Arrange
            string url = "http://www.google.com";

            // Act
            string result = _parser.ParseHostName(url);

            // Assert — Single Assertion Rule
            Assert.That(result, Is.EqualTo("google.com"));
        }

        [Test]
```

```

        public void ParseHostName_HttpsUrlWithWwwPrefix_ReturnsHostWithoutWww()
        {
            string url    = "https://www.example.com";
            string result = _parser.ParseHostName(url);

            Assert.That(result, Is.EqualTo("example.com"));
        }

        [Test]
        public void ParseHostName_UrlWithWwwAndPath_ReturnsHostOnlyWithoutWww()
        {
            string url    = "http://www.google.com/search?q=nunit";
            string result = _parser.ParseHostName(url);

            Assert.That(result, Is.EqualTo("google.com"));
        }

        // Execution Path 2: URL without www. prefix

        [Test]
        public void ParseHostName_UrlWithoutWwwPrefix_ReturnsHostAsIs()
        {
            // Arrange
            string url = "http://google.com";

            // Act
            string result = _parser.ParseHostName(url);

            // Assert
            Assert.That(result,
            Is.EqualTo("google.com"));
        }

        [Test]
        public void ParseHostName_HttpsUrlWithoutWwwPrefix_ReturnsHostAsIs()
        {
            string url    = "https://api.example.com";
            string result = _parser.ParseHostName(url);

            Assert.That(result, Is.EqualTo("api.example.com"));
        }

        [Test]
        public void ParseHostName_UrlWithoutWwwAndWithPath_ReturnsHostOnly()
        {
            string url    = "https://portal.cognizant.com/home/dashboard";
            string result = _parser.ParseHostName(url);

            Assert.That(result, Is.EqualTo("portal.cognizant.com"));
        }

        // Edge cases

        [Test]
        public void ParseHostName_UrlWithNoScheme_ReturnsInputAsIs()
        {
            // No "://" present - returns as-is (no scheme to strip)
            string url    = "www.google.com";
            string result = _parser.ParseHostName(url);

            Assert.That(result, Is.EqualTo("google.com"));
        }

        [Test]
        public void ParseHostName_UrlWithNoSchemeAndNoWww_ReturnsInputAsIs()
        {
            string url    = "google.com";
            string result = _parser.ParseHostName(url);

```

```

        Assert.That(result, Is.EqualTo("google.com"));
    }

    [Test]
    public void ParseHostName_ResultIsNotNull()
    {
        string url = "http://www.example.com";
        string result = _parser.ParseHostName(url);

        Assert.That(result, Is.Not.Null);
    }

    [Test]
    public void ParseHostName_ResultIsNotEmpty()
    {
        string url = "http://www.example.com";
        string result = _parser.ParseHostName(url);

        Assert.That(result, Is.Not.Empty);
    }

    [Test]
    public void ParseHostName_ResultDoesNotContainWwwDot()
    {
        string url = "http://www.cognizant.com";
        string result = _parser.ParseHostName(url);

        // Regardless of path, result must never contain "www."
        Assert.That(result, Does.Not.Contain("www."));
    }

    [Test]
    public void ParseHostName_ResultDoesNotContainScheme()
    {
        string url = "https://www.example.com";
        string result = _parser.ParseHostName(url);

        Assert.That(result, Does.Not.Contain("://"));
    }
}

```

Step 4 — Run Tests (All Pass)

Run from Visual Studio

```

# Menu: Test → Run All Tests
# or keyboard shortcut: Ctrl + R, A

```

```

# Alternatively via CLI:
dotnet test

```

Test Explorer Output — All Green

```

Test Run started... -----
Test Explorer - UtilLib.Tests

✓ ParseHostName_UrlWithWwwPrefix_ReturnsHostWithoutWww
✓ ParseHostName_HttpsUrlWithWwwPrefix_ReturnsHostWithoutWww
✓ ParseHostName_UrlWithWwwAndPath_ReturnsHostOnlyWithoutWww
✓ ParseHostName_UrlWithoutWwwPrefix_ReturnsHostAsIs
✓ ParseHostName_HttpsUrlWithoutWwwPrefix_ReturnsHostAsIs
✓ ParseHostName_UrlWithoutWwwAndWithPath_ReturnsHostOnly ✓
ParseHostName_UrlWithNoScheme_ReturnsInputAsIs
✓ ParseHostName_UrlWithNoSchemeAndNoWww_ReturnsInputAsIs
✓ ParseHostName_ResultIsNotNull

```

```

✓ ParseHostName_ResultIsNotEmpty
✓ ParseHostName_ResultDoesNotContainWwwDot ✓
  ParseHostName_ResultDoesNotContainScheme
-----Test
Run Successful. Total tests: 12   Passed: 12   Failed:
0   Skipped: 0
Duration: 0.54 s

```

Step 5 — Break the Source Code and Rerun Tests

Modify ParseHostName in UriHostNameParser.cs to Introduce a Bug

We remove the `www.` stripping logic (Path 1) to simulate a defect.

```

// BROKEN version - Execution Path 1 removed intentionally to break tests

public string ParseHostName(string url) {
    string withoutScheme = url;

    if (url.Contains("://"))
    {
        withoutScheme = url.Substring(url.IndexOf("://") + 3);
    }

    int slashIndex = withoutScheme.IndexOf('/');
    if (slashIndex >= 0) {
        withoutScheme = withoutScheme.Substring(0, slashIndex);
    }

    // BUG: www. stripping block removed - method now always returns host as-is
    // if (withoutScheme.StartsWith("www."))
    // {
    //     return
    withoutScheme.Substring(4); // }

    return withoutScheme; // returns "www.google.com" instead of "google.com" }

```

Rerun Tests — Failures Expected

```

Test Run started... -----
Test Explorer - UtilLib.Tests

X ParseHostName_UrlWithWwwPrefix_ReturnsHostWithoutWww
  Expected: "google.com"
  But was:  "www.google.com"

X ParseHostName_HttpsUrlWithWwwPrefix_ReturnsHostWithoutWww
  Expected: "example.com"
  But was:  "www.example.com"

X ParseHostName_UrlWithWwwAndPath_ReturnsHostOnlyWithoutWww
  Expected: "google.com"
  But was:  "www.google.com"

X ParseHostName_ResultDoesNotContainWwwDot
  Expected: string not containing "www."
  But was:  "www.cognizant.com"

✓ ParseHostName_UrlWithoutWwwPrefix_ReturnsHostAsIs
✓ ParseHostName_HttpsUrlWithoutWwwPrefix_ReturnsHostAsIs
✓ ParseHostName_UrlWithoutWwwAndWithPath_ReturnsHostOnly ✓
  ParseHostName_UrlWithNoScheme_ReturnsInputAsIs
✓ ParseHostName_UrlWithNoSchemeAndNoWww_ReturnsInputAsIs
✓ ParseHostName_ResultIsNotNull
✓ ParseHostName_ResultIsNotEmpty
✓ ParseHostName_ResultDoesNotContainScheme

```

```
-----Test
Run Failed. Total tests: 12   Passed: 8   Failed: 4
Skipped: 0
Duration: 0.49 s
```

Failed tests correctly caught the defect in Execution Path 1.
The tests that cover Path 2 (no www.) still pass – they are unaffected.

This demonstrates the value of unit tests: - Removing the www. logic broke 4 tests immediately. - Without tests, this defect would have reached production undetected. - The test names clearly identify WHICH scenario failed and WHY.

Step 6 — Restore the Source Code and Rerun

Restore the www. Stripping Logic in ParseHostName

```
// Restore Execution Path 1 – put back the www. stripping block

if (withoutScheme.StartsWith("www.")) {
    return withoutScheme.Substring(4); }
```

Final Test Run — All Green Again

```
Test Run started...

----- ✓
ParseHostName_UrlWithWwwPrefix_ReturnsHostWithoutWww
✓ ParseHostName_HttpsUrlWithWwwPrefix_ReturnsHostWithoutWww
✓ ParseHostName_UrlWithWwwAndPath_ReturnsHostOnlyWithoutWww
✓ ParseHostName_UrlWithoutWwwPrefix_ReturnsHostAsIs
✓ ParseHostName_HttpsUrlWithoutWwwPrefix_ReturnsHostAsIs
✓ ParseHostName_UrlWithoutWwwAndWithPath_ReturnsHostOnly ✓
ParseHostName_UrlWithNoScheme_ReturnsInputAsIs
✓ ParseHostName_UrlWithNoSchemeAndNoWww_ReturnsInputAsIs
✓ ParseHostName_ResultIsNotNull
✓ ParseHostName_ResultIsNotEmpty
✓ ParseHostName_ResultDoesNotContainWwwDot ✓
ParseHostName_ResultDoesNotContainScheme

-----Test
Run Successful. Total tests: 12   Passed: 12   Failed:
0   Skipped: 0
Duration: 0.51 s
```

NUnit Attributes Used — Quick Reference

```
[TestFixture] // Marks the class as a container of NUnit tests
[SetUp]       // Runs before EACH test – used to create a fresh _parser instance
[Test]        // Marks an individual test method
[OneTimeSetUp] // Runs ONCE before all tests (used when setup is expensive)
[TestCase]    // Parameterised test – passes inline arguments to the method
[TearDown]    // Runs after EACH test – clean up resources

// Assert.That() patterns used:
Assert.That(result, Is.EqualTo("expected")); // equality Assert.That(result,
Is.NotNull()); // not null
Assert.That(result, Is.Not.Empty); // not empty string
Assert.That(result, Does.Not.Contain("www.")); // string does not contain
Assert.That(result, Does.Not.Contain(":/")); // string does not contain scheme
```

Single Assertion Rule: each [Test] method contains exactly one Assert.That(). This makes it immediately clear which specific behaviour failed when a test turns red.